YAQZA

Predictive Maintenance System for Turbofan Engines

Digital Egypt Pioneers Initiative (DEPI)
Graduation Project Report

Version: 1.0.0
Date: July 2026

Project Team

Aya Yasser

Basmala Saeed

Esraa Mohamed

Ahmed Badawy

Omar Abdullah

Table of Contents

Executive Summary

The YAQZA project introduces an advanced Predictive Maintenance (PdM) System specifically designed for commercial turbofan engines. By leveraging the NASA CMAPSS dataset, the system utilizes a robust 5-model Machine Learning ensemble to predict the Remaining Useful Life (RUL) of aircraft engines with a prediction error of just 15 cycles. This capability allows airlines and Maintenance, Repair, and Overhaul (MRO) organizations to transition from reactive and fixed-interval maintenance to a proactive, data-driven approach, potentially saving up to $450,000 per prevented emergency and ensuring maximum passenger safety. The system is enhanced by a Gemini-powered bilingual AI advisor to deliver actionable maintenance reports in English and Arabic.

Introduction

Background

The aviation industry relies heavily on turbofan engines, which operate under extreme mechanical and thermal stress. Traditional maintenance scheduling relies on static cycle thresholds, leading to premature part replacements or unexpected in-flight failures.

Problem Statement

Unscheduled engine failures result in massive logistical costs, grounded aircraft, and severe safety risks. Conversely, fixed maintenance schedules waste functional component life. The "Compensation Trap"—where an engine burns more fuel to hide degrading efficiency—makes single-sensor monitoring ineffective.

Motivation

By accurately forecasting engine degradation over 60 flight cycles in advance, airlines can schedule maintenance optimally, ensuring zero unscheduled removals and maximizing component lifespans.

Objectives

- Predict turbofan RUL with an RMSE of ≤ 15 cycles.
- Integrate 21 sensor readings across 6 operating conditions.
- Provide a scalable API for real-time SCADA/IoT ingestion.
- Deliver automated, bilingual AI maintenance recommendations.

Scope & Out of Scope

Scope: The project covers data ingestion, preprocessing, predictive modeling via an ensemble approach, a REST API backend, a dashboard frontend, and AI-driven reporting.

Out of Scope: Physical hardware deployment on aircraft, integration with proprietary OEM closed systems, and maintenance execution (physical repairs).

Chapter 1: Project Planning

Project Overview

YAQZA is a full-stack SaaS solution targeting regional airlines and MROs, offering tiered subscriptions for predictive engine analytics.

Goals

- **Business:** Reduce emergency maintenance costs by 90%. Achieve a positive ROI for clients within the first year.
- **Technical:** Implement a 5-model consensus architecture. Maintain API response times under 500ms. Ensure 99.9% system uptime with auto-scaling Kubernetes deployment.

Timeline & Methodology

- **Month 1:** Data exploration, CMAPSS EDA, baseline modeling.
- **Month 2:** Feature engineering, ML ensemble development, API design.
- **Month 3:** Frontend development, Gemini AI integration, database setup.
- **Month 4:** Testing, deployment, CI/CD pipeline creation, documentation.

The team utilized Agile/Scrum methodology, conducting weekly sprint planning and stand-ups to ensure continuous integration and iterative improvement.

Risk Management: Data quality issues are handled via rigorous validation schemas in the API. System failures are addressed with a 4-level graceful degradation strategy and Redis caching.

Chapter 2: Stakeholder Analysis

Stakeholder	Role	Expectations	Influence	Interest
Airlines/MROs	Client	Cost reduction, system reliability	High	High
Maintenance Crew	End Users	Clear actionable insights, bilingual reports	Medium	High
Project Team	Creators	Successful deployment, graduation	High	High
DEPI Instructors	Supervisors	Academic rigor, technical excellence	High	Medium

Chapter 3: Requirements Analysis

Functional & Non-functional Requirements

- **Functional:** The system must ingest JSON payloads of 21 sensor readings, predict RUL using the 5-model ensemble, and generate maintenance reports in Arabic and English.
- **Performance:** Inference must complete in under 100ms.
- **Availability:** RTO < 1 hour, RPO < 6 hours.
- **Security:** API Key authentication and TLS 1.3 encryption.

Business Rules: If a database failure occurs, the system must automatically degrade to heuristic-based rule evaluation rather than failing entirely.

Chapter 4: System Analysis

The operational workflow follows a strict, sequential pipeline from data ingestion to user presentation:

1. **Ingestion:** IoT sensors send readings to the API gateway via structured HTTP POST requests.
2. **Preprocessing:** Data is dynamically scaled per operating condition. Baselines are removed to establish relative degradation signatures.
3. **Feature Extraction:** A 30-cycle rolling window is applied. TSFresh extracts features, FDR filters out irrelevant ones, and PCA compresses the dimensions.
4. **Prediction:** Processed features concurrently feed into Ridge, RF, XGBoost, NGBoost, and CNN-LSTM models.
5. **Consensus:** A weighted average RUL and probabilistic uncertainty bound are calculated to provide a final decision.
6. **Reporting:** The AI layer intercepts the outputs to generate specific human-readable maintenance actions.

Chapter 5: System Architecture

YAQZA operates on a microservices-inspired architecture deployed on Kubernetes, guaranteeing scalability and high availability.

- **Frontend:** React.js dashboard communicating via REST APIs.
- **Backend/API:** FastAPI acting as the core gateway, handling routing and authentication.
- **ML Serving:** ONNX runtime hosting pre-trained models for sub-100ms inference.
- **Data Layer:** PostgreSQL for cloud persistence, SQLite for edge deployment, Redis for caching, and Amazon S3 for periodic backups.
- **AI Layer:** External API integrations via Gemini 2.5 Flash for advanced NLP and translation tasks.

Chapter 6: Database Design

The relational database serves as the single source of truth for engine metadata, telemetry, and prediction logs.

Core Tables

- `engines`: Stores engine metadata (`engine_id` PK, `model`, `date_commissioned`).
- `telemetry`: Stores raw cycle readings (`reading_id` PK, `engine_id` FK, `cycle`, and 21 numerical sensor columns).
- `predictions`: Stores ML system outputs (`pred_id` PK, `engine_id` FK, `timestamp`, `predicted_rul`, `confidence_score`).

Chapter 7: Backend Design

The backend is engineered with FastAPI for high-performance, asynchronous request handling.

- `backend/main.py`: The primary entry point. Initializes the FastAPI app, configures CORS, and registers API routers.
- `backend/features.py`: Encapsulates the 8-step ML pipeline (KMeans scaling, PCA transformation, rolling windows) applying them to incoming live streams.
- `backend/database.py`: Manages SQLAlchemy session binding and connection pooling mechanisms.
- `backend/models.py`: Defines SQLAlchemy ORM models corresponding to the underlying database tables.
- `backend/crud.py`: Abstracted Create, Read, Update, Delete functions for safe database interaction.
- `backend/schemas.py`: Defines strict Pydantic models to ensure absolute request/response payload validation.
- `backend/gemini_advisor.py`: Wraps the HTTP client calls to the Gemini API, formulating prompts and parsing bilingual outputs.
- `backend/seed_data.py`: A utility script to quickly populate the database with CMAPSS test data for staging and development.

Chapter 8: Machine Learning

The system was trained utilizing the NASA CMAPSS dataset, which simulates realistic turbofan degradation under operational stress.

Preprocessing Pipeline

- **Scaling:** Because sensor readings shift drastically across varying altitudes and Mach numbers, K-Means clustering is utilized to categorize the 6 distinct operating conditions. A standard scaler is then applied per cluster.
- **Rolling Window:** To capture temporal sequential patterns, a 30-cycle sliding window is mapped over the data streams.
- **Feature Selection:** TSFresh is deployed to generate statistical and frequency features. False Discovery Rate (FDR) hypothesis testing eliminates features with a p-value > 0.05. Finally, PCA reduces dimensionality while retaining 95% of the predictive variance.

Chapter 9: Model Documentation

Model	Description	Why Selected	Expected RMSE
Ridge	Linear baseline with L2 regularization.	Sanity check, extremely fast inference.	~20
Random Forest	Bagging ensemble of decision trees.	Captures non-linear relationships, highly robust.	~17
XGBoost	Gradient boosting sequential tree model.	Provides best tabular performance and sequential error correction.	15
NGBoost	Probabilistic gradient boosting.	Provides critical uncertainty quantification and confidence intervals.	16
CNN-LSTM	Deep learning hybrid architecture.	Excels at capturing complex spatiotemporal degradation patterns.	18

Chapter 10 & 11: Frontend & UI/UX Design

The React frontend features a sophisticated, dark-themed interface specifically tailored for aviation engineering teams minimizing cognitive strain in low-light MRO environments.

- **Overview Dashboard:** Displays real-time fleet status, active prognostic alerts, and aggregate health metrics.
- **Engine Diagnostics:** Renders interactive time-series charts mapping 21-sensor degradation curves against predicted RUL.
- **Color Palette:** Employs professional deep blues and slate grays, reserving strict traffic-light colors (Red/Yellow/Green) exclusively for actionable status indicators.

Chapter 12: API Documentation

Key Endpoints

- **POST /ingest**

  Purpose: Receive real-time telemetry from SCADA/IoT systems.

  Body: JSON array containing `engine_id`, `cycle`, and 21 sensor float values.

- **GET /predict/{engine_id}**

  Purpose: Retrieve the ensemble consensus RUL prediction.

  Response: `{"engine_id": "123", "rul": 45, "status": "WARNING"}`

- **GET /analyze/{engine_id}**

  Purpose: Trigger the Gemini AI for bilingual maintenance reporting.

  Response: JSON payload containing `report_en`, `report_ar`, and structured action items.

Chapter 13: AI Integration

The system seamlessly integrates Gemini 2.5 Flash as an intelligent AI Advisor. When the `/analyze` endpoint is called, the backend compiles the latest 30 operational cycles, the ensemble RUL, and the corresponding confidence bounds into a structured semantic prompt. Gemini subsequently generates a plain-language summary of the engine's current health status, identifies anomalous sensor trends (e.g., "HPC Outlet Temp is rising abnormally"), and formulates specific maintenance recommendations formatted in both professional English and standard Arabic.

Chapter 14: Testing

Comprehensive testing ensures system reliability under mission-critical conditions:

- **Unit Testing:** Utilizing `pytest` to rigorously validate data transformations within `features.py`.
- **API Testing:** The FastAPI `TestClient` is employed to validate endpoint responses, ensuring proper `400 Bad Request` handling on invalid sensor formats.
- **ML Validation:** K-fold cross-validation performed across FD001-FD004 CMAPSS datasets to prevent overfitting.
- **Disaster Recovery:** Simulated database drops were conducted to verify the 4-level graceful degradation framework (successfully falling back to Redis caching and heuristic rules without dropping API connections).

Chapter 15: Team Contributions

Member Name	Role	Responsibilities	Files / Modules
Aya Yasser	ML Engineer	Deep Learning, Ensemble Logic	`models.py`, CNN-LSTM notebook
Basmala Saeed	Backend Developer	API Gateway, Database Design	`main.py`, `database.py`, `crud.py`
Esraa Mohamed	Data Scientist	EDA, Preprocessing, XGBoost	`features.py`, XGBoost notebook
Ahmed Badawy	Frontend Dev	React Dashboard, UI/UX	Frontend repository, Visualizations
Omar Abdullah	Cloud Engineer	Deployment, CI/CD, DevOps	Dockerfiles, K8s manifests

Chapter 16: Challenges & Solutions

- **Challenge (The Compensation Trap):** Sensor values represented vastly different engine states at varying altitudes. An engine burning more fuel to mask degradation appeared healthy to basic thresholds.
  Solution: Engineered condition-specific K-Means clustering prior to scaling, which successfully standardized the baseline regardless of the flight phase.
- **Challenge (Tabular Data Complexities):** Deep Learning architectures (CNN-LSTM) unexpectedly underperformed compared to Tree-based models on tabular sensor readings.
  Solution: Shifted primary inferential weight to XGBoost, retaining CNN-LSTM strictly within the ensemble configuration to capture rare edge-case spatiotemporal anomalies.

Chapter 17: Future Enhancements

- Integration of high-frequency physical vibration data (accelerometers) alongside existing thermodynamic sensors.
- Deployment of lightweight ONNX models directly to aircraft edge computing units for in-flight offline inference.
- Support for fully immersive 3D Digital Twin visualizations within the frontend dashboard.

Chapter 18: Conclusion

YAQZA demonstrates the profound technological and financial impact of merging traditional machine learning ensembles, deep learning, and Generative AI within the aviation sector. By achieving an RUL prediction error of just 15 cycles, the system empowers airlines to transition aircraft maintenance from a reactive, unpredictable liability into a proactive, strictly controlled process. The robust backend architecture ensures enterprise-grade reliability, directly fulfilling the core business objectives of maximizing safety while minimizing operational downtime and costs.

References

1. NASA Commercial Modular Aero-Propulsion System Simulation (CMAPSS) Dataset, Glenn Research Center.
2. FastAPI Documentation Framework, *https://fastapi.tiangolo.com/*
3. XGBoost: A Scalable Tree Boosting System, Chen & Guestrin.
4. NGBoost: Natural Gradient Boosting for Probabilistic Prediction, Duan et al.

Appendix

Project Structure (Partial)

```
yaqza/
├── backend/
│   ├── main.py
│   ├── features.py
│   ├── database.py
│   ├── models.py
│   ├── crud.py
│   ├── schemas.py
│   ├── gemini_advisor.py
│   └── seed_data.py
├── notebooks/
└── frontend/
```

Environment Variables (.env)

```
DATABASE_URL=sqlite:///./yaqza.db
REDIS_URL=redis://localhost:6379
GEMINI_API_KEY=your_api_key_here
PORT=8000
```